

CodeAtlas 码鉴：证据驱动的 AI 代码审查与项目体检工作台

比赛技术文档 · 2026 年 9 月 · v0.1.0 · 公开仓库 github.com/aabbcc070403/codeatlas · 技术栈 Next.js 15 + React 19 + TypeScript (strict) + PostgreSQL/pgvector + 独立 Node worker

CodeAtlas 码鉴面向高校学生与缺少专职审查人员的小型前端团队：用户上传 ZIP 代码包，系统先运行确定性的静态规则体检，再在严格预算与工具约束下进行 AI 定向审查，最终把每一条问题连同**可点击核验的路径、行号与引文证据**呈现出来，并支持受限追问、单文件补丁提案、三格式报告导出与快照对比。

本文档按「产品与功能 → 系统架构 → AI 工作流 → 评测实验 → 工程质量 → 创新差异 → 部署使用」组织。文中所有评测数字均来自仓库内文档记录的真实命令运行，可回溯；所有 AI 相关结果均如实标注其运行环境——**真实模型评测已于 2026-09-19 执行**（DeepSeek `deepseek-chat`，holdout 每模式重复 3 次，数字见 5.5 节），2026-09-13 的 llm 模式数字为受控 **Mock 管线**验证（结果逐项标注 `provider=mock`），两套数字分栏并存、互不覆盖；本地演示 AI 默认 `AI_PROVIDER=mock`，界面全程标注。

核心功能交付 F01–F12 全部落地 规格 P0 范围 12/12	自动化测试 332 项全绿 单元 197 · 集成 127 · E2E 8
确定性静态规则 9 条 AST 规则 超过规格要求的 6 条	评测数据集 24 个样例项目 12 缺陷 + 12 对照 · 6 类问题

1. 项目概述与定位

1.1 问题背景：三类工具都回答不了「证据在哪」

正在完成课程设计、竞赛项目或首个 Web 产品的学生，以及小型前端团队，普遍缺少专职代码审查人员。他们可用的工具各有断层：**规则工具**（通用 linter）能指出部分缺陷，但解释不贴近项目上下文，初学者难以判断优先级；**通用聊天工具**需要手工整理代码、粘贴上下文，模型给出的引用难以核实，也无法验证修改是否有效；**把整仓发给模型**则成本高、覆盖不透明、输出不稳定。三类方案都无法回答同一个追问：这条结论的证据在哪、我该先修哪一个。

1.2 产品的回答：确定性打底，AI 定向，证据收口

CodeAtlas 把「项目体检」拆成三层递进的设计：

- **确定性静态规则打底**——9 条基于 TypeScript Compiler API 的 AST 规则（动态执行、HTML 注入入口、postMessage 源、JSX key、条件 Hook、检查禁用等），零模型成本、结果可复现；上下文不足的命中一律标 `needs_review`（待核查），不把「危险 API 出现」直接判为可利用漏洞。
- **受预算约束的 AI 定向审查**——按风险优先抽样文件，AI 只能经四件只读工具读取当前快照与规范库，所有输出必须通过服务端独立的证据校验（详见第 4 章）。
- **证据工作台收口**——每条问题带路径 / 行号 / 引文，点击即定位代码行；用户可确认或标记误报（风险指数联动重算），可围绕问题追问、生成可审阅的单文件补丁提案，并可导出报告、对比两次快照的问题变化。

诚实边界（产品原则）。产品不宣称「证明代码安全」「发现全部 Bug」或「自动修复成功」。静态命中、AI 推断、语法检查与真实测试结果始终使用不同标签展示；Mock 与真实模型输出在界面、报告、评测中全程可辨；「未再检出 ≠ 已验证修复」。本文档延续同一原则：未执行的验证一律如实列为限制，不虚构数字。

2. 核心功能与使用流程

2.1 功能总览（对应规格 F01–F12）

功能	交付内容
F01 项目与导入	项目创建、ZIP 导入（路径穿越 / 压缩炸弹 / 符号链接防护）、快照历史、文件树、只读代码预览
F02 结构体检	文件统计、语言分布、依赖声明概览、忽略明细（逐条原因）、相对导入关系
F03 静态规则	9 条确定性 AST 规则；候选问题标 <code>needs_review</code> ，行重叠去重
F04 AI 审查	风险优先抽样 + 受限工具循环；结构化结果全部经证据校验
F05 专属 RAG	30 条预置版本化规范 + 项目自有 Markdown 导入；pgvector + 字法双路 RRF 融合检索
F06 证据工作台	筛选、代码行定位、来源、触发条件、影响、修复建议、确认 / 误报反馈
F07 受限追问	围绕当前问题与快照的追问（4 次模型 / 8 次工具 / 60 秒）；无终端、联网、写文件工具
F08 补丁提案	单问题单文件 diff；旧文本 / 哈希 / 脱敏区间 / 语法四重校验；下载 <code>.patch</code> ，不写入原件
F09 报告导出	JSON / Markdown / 可打印 HTML 三格式，与界面同一数据源
F10 快照对比	新增 / 仍存在 / 未再检出 / 不可比较四类变化；「未再检出 ≠ 已验证修复」固定说明

功能	交付内容
F11 评测中心	24 样例 / 6 类问题 / 16-8 划分数数据集；static_only / llm_no_rag / hybrid_rag 三模式指标与消融
F12 演示保障	demo / admin 两级访问码、会话隔离、任务取消、租约恢复、TTL 过期清理、日 token 预算

功能与规格逐条对应；实现与验证记录见仓库 `docs/progress.md`（T01–T08 与 R01–R09 全部任务含命令退出码）。

2.2 端到端使用流程

以下主线同时也是 5–8 分钟演示脚本（`docs/demo-script.md`）的骨架，每一步都有真实代码路径支撑：

- 导入**——访问码登录进入工作台，新建项目并上传 ZIP。导入摘要展示文件数、可分析文件数、忽略列表及逐条原因、脱敏记录，并明确提示「用户主动选择云端 AI 才发送代码片段」。防护在服务端：路径穿越 / 绝对路径 / 盘符 / 符号链接全拒绝，按流式实际解压字节限额（压缩 20MiB / 解压 80MiB / 2000 条目 / 单文件 512KiB），`.env` 与私钥等敏感文件拒绝收录，疑似密钥等长遮盖后参与后续流程。
- 静态体检**——点击「开始审查」，扫描页实时展示六阶段进度（`ingest` → `index` → `static` → `ai` → `validate` → `report`）与事件流；完成后呈现风险指数卡片（`min(100, 20×critical + 10×high + 4×medium + low)`），**待核查不计入**）、静态覆盖与问题列表。
- 证据查看**——问题详情页三栏布局：左侧问题列表，中间只读代码（行号 1 起始、主引用高亮、引用点击定位），右侧证据面板（触发条件 / 影响 / 依据 / 建议，规范引用的标题、版本与来源链接）。点「确认问题」或「标记误报」，风险指数即时联动更新。
- AI 定向读取与追问**——扫描时勾选云端 AI 才进入 `ai` 阶段；扫描页「工具轨迹」面板展示每次工具调用的名称、输入摘要、真实耗时与状态（不展示思维链）。问题详情页的追问面板围绕当前问题 + 当前快照回答，带 Mock / 真实模型标注、模型与工具调用计数、检索模式；证据不足时明确返回 `insufficient_evidence`，不凭空断言。
- 补丁提案**——单文件 diff 逐行着色，可应用 / 语法（基线 → 补丁后错误数比较） / 测试三项验证状态分开呈现，测试恒为 `not_run`；点「下载」获得 `.patch` 附件，上传原件字节不变（集成测试断言）。
- 对比**——上传修复版 ZIP 再次审查后，「快照对比」页给出四类变化（新增 / 仍存在 / 未再检出 / 不可比较）与逐条列表，并固定展示「未再检出 ≠ 已验证修复」。

3. 系统架构

3.1 技术栈与运行形态

单包工程：Next.js 15.5.25 (App Router) + React 19.3 + TypeScript 5.9 strict 前端与服务端同构；Tailwind CSS + shadcn/ui 风格组件 + Lucide 图标 + CodeMirror 6 (只读代码) + ECharts (风险 / 依赖图)；数据层为 PostgreSQL 协议数据库 + Drizzle ORM + 自研迁移器——本地开发用嵌入式 PGlite 0.5.8 (含真实 pgvector 扩展) 经自研 Postgres 线协议 socket 服务暴露，生产为 PostgreSQL 16 + pgvector，两者同构；后台为**独立 Node worker** (无 shell 工具)，作业经数据库 `FOR UPDATE SKIP LOCKED` 租约领取，**不引入 Redis**，不用 Serverless 承载长扫描。AI 侧用 AI SDK v5 对接 OpenAI 兼容 Chat Completions 接口，模型输入输出经 Zod 合同校验。测试为 Vitest (单元 / 集成) + Playwright (E2E)。

3.2 总体模块与数据流

数据库是唯一协调点：web 只创建任务行并读取状态与结果，worker 持久化执行，浏览器经 SSE / 轮询消费事件。



图 1: 总体模块与数据流。

web 与 worker 通过数据库解耦；所有扫描事件持久化为 SSE 游标，刷新与断线均从持久化状态恢复。

3.3 数据流与关键机制

- **导入 (ingest)** ——yauzl 流式解压 + 实际解压字节限额；路径校验覆盖穿越 / 绝对路径 / 盘符 / UNC / 保留设备名 / ADS 冒号 / NUL / 大小写冲突 (/ 与 \ 双分隔符识别)，ZIP 内嵌 ZIP 不递归；等长脱敏 (赋值密钥 / AKIA / ghp_ / sk- / Alza / JWT / PEM 块，保持行号与字符位置) 并记录 redactedRanges；生成不可变快照 (内容哈希、CRLF→LF 与 BOM 标准化、UTF-8 严格解码) 与结构统计；忽略策略逐条记录原因并在界面展示。
- **六阶段扫描管线**——ingest (快照核对) → index (自有内存 TS 程序解析 AST，不加载项目插件；Vue SFC script 块抽取 + 原始行号映射) → static (9 条规则，候选标 needs_review) → ai (用户勾选云端 AI 且 chat 就绪时) → validate (对全部 finding 的引文与快照真实行严格比对，无效删除并计数) → report (结果、风险指数、覆盖、usage 同事务终态化)。事件持久化到 scan_events (全局递增 bigint 即 SSE 游标)；取消在阶段边界检查；崩溃恢复从已持久化的 scan.stage 继续，事件按 scan_events 查重不重发，findings 按 fingerprint 幂等。

- **租约与生命周期**—— `FOR UPDATE SKIP LOCKED` 原子领取； `LeaseKeeper` 每 10s 续 30s 租约，DB 连续异常 3 次才判定失租（防抖动）；阶段推进、事件、终态全部在 `leaseGuarded` 事务内（锁定任务行并校验 owner / generation / running / 未过期），**失租不能写任何结果**；上限耗尽时任务与目标状态一致终结，`reapExhaustedJobs` 收割崩溃残留；项目删除先请求取消任务并写 `deleting_at`，先删存储再删 DB 行；TTL 默认 24 小时，worker 每小时清理。
- **SSE 与轮询降级**—— `GET /api/scans/:id/events` 支持断线后从 `Last-Event-ID` 续传持久化事件，15s 心跳；客户端连续失败时降级为 2s 起步、逐次退避至 10s 封顶的轮询，恢复后停止；刷新页面从持久化结果恢复，不依赖内存状态。
- **数据库合同**——16 张业务表 + `daily_usage` 预算表；关键唯一约束 `files(snapshotId,path)`、`jobs(kind,targetId)`、`scans(snapshotId,idempotencyKey)`、`findings(scanId,fingerprint)`；三条迁移由自研迁移器（journal + 内容哈希幂等、单事务 + advisory 锁）管理，不改写已应用迁移。

3.4 关键工程决策（实测细节）

自研 PGLite 线协议 socket 层。官方 `@electric-sql/pglite-socket` 按「单条协议消息」跨连接排队，两个客户端的 Parse/Bind 交错会互相覆盖共享 unnamed prepared statement（实测 2000 次并发出现 475 次报错）。自研实现以「完整查询批次」为最小执行单元，并为 unnamed statement 分配会话独占名（pgbouncer 同款方案），修复后 6 客户端 3000+ 混合查询零错误。该层同时兼容 jsonb 读回为字符串的 OID 限制（`asJson` 助手统一处理）。

Next 15 构建崩溃定位与规避。内置 nft 对 `@vue/compiler-sfc` 静态求值时把 `mock` 的 `path.join` 当 `thenable` 调用导致 build 崩溃（最小复现确认仅该文件触发）；经 `outputFileTracingExcludes` 排除后构建通过。代价是 standalone 产物 trace 不含该依赖，故 Dockerfile 采用完整 `node_modules` + `next start` 部署并在注释中写明缘由。

4. AI 工作流深度解析

本章对应评审关注的核心：AI 不是「调用一次大模型」，而是一条**可预算、可校验、可追溯**的受控工作流。技术路线上明确不采用「整仓发送给模型」（成本高、覆盖不透明、输出不稳定）与「多智能体自主修改仓库」（需执行沙箱与昂贵调度），首版为**静态规则 + 有限步 AI 工具循环 + RAG**的编排器形态。

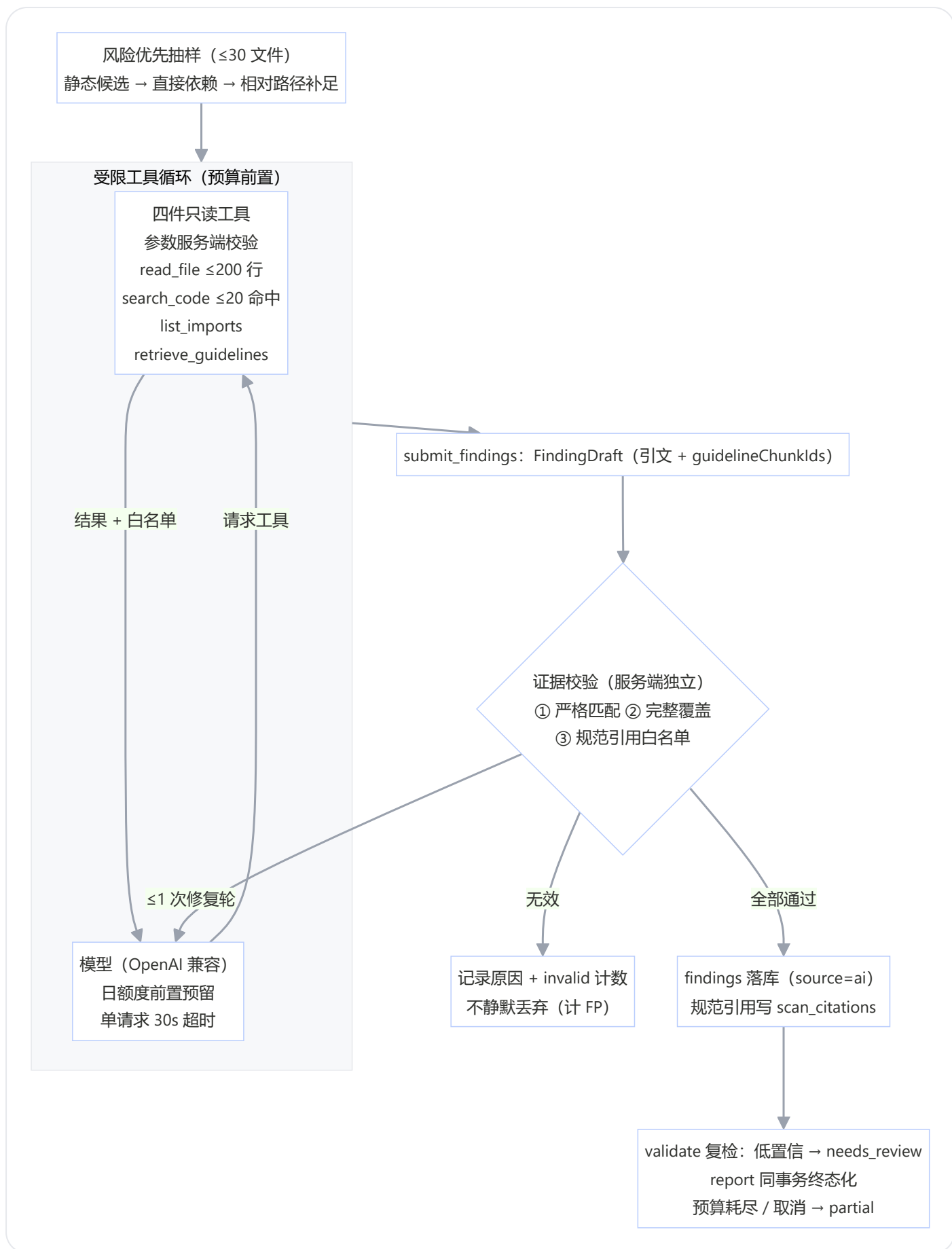


图 2: AI 审查 workflow。

抽样 → 受限工具循环 → 结构化提交 → 服务端证据校验 → 修复轮 / 无效计数 → 终态化；每个环节的预算与校验都在服务端强制执行。

4.1 风险优先抽样与覆盖诚实

AI 阶段不整仓发送：先取静态候选文件，再取其直接依赖，最后按相对路径排序补足至 30 个文件；任务中额外读取的文件纳入实际 AI 覆盖集合。静态覆盖与 AI 覆盖**单独计数**，覆盖记录包含选择依据、读过的行段、未读文件与预算限制——不以「已索引」冒充「已审查」。

4.2 受限工具循环：四件只读工具

模型仅有的行动能力是四件只读工具，参数全部由服务端校验，不接受任意 URL、SQL、命令与绝对路径：

工具	服务器侧约束
<code>read_file</code>	读取当前快照指定路径的行段，单次 ≤ 200 行
<code>search_code</code>	字面量检索（非正则），最多 20 个命中
<code>list_imports</code>	仅返回已解析的导入关系
<code>retrieve_guidelines</code>	规范库检索， $\text{topK} \leq 8$ ；实际返回的 <code>chunkId</code> 构成引用白名单

无终端、联网、写文件工具；模型输出经 Zod 校验；系统提示明确禁止代码注释、README、规范文档等不可信数据扩大工具权限（指令注入防护）。

4.3 双路 RAG 检索 (F05)

- **知识库**——30 条自写规范摘要（安全 / 正确性 / 性能 / 可维护性，来源 MDN / OWASP / react.dev，附版本与 contentHash）；seed 幂等（builtinKey + contentHash，内容变化版本 +1）；支持导入项目自有 Markdown，按标题 / 段落切块（约 400–800 字符），保留文档名、标题与起止行。
- **双路检索**——pgvector 余弦检索 + Unicode 字符二元组词法检索两路各取 Top-N，经 RRF（ $k=60$ ）融合取前 5；中文检索不依赖 PostgreSQL 英文分词器。向量查询按 `embedding_model` 筛选，不同模型 / 维度的旧向量不混用语义空间；嵌入不可用时降级为词法检索并在结果中标注 `lexical_only`。
- **引用可溯**——每条 finding 的规范引用写入 `scan_citations`（版本 + 文本快照 + 来源链接），知识库后续变更后旧报告仍可解释当时依据的来源。

4.4 证据校验：可信性的核心

AI 结论在落库前后经过两道同一实现的服务端校验（提交时 validator 与扫描 validate 阶段复用同一代码，消除两层标准）：

- **引文校验**——统一 LF 后**逐字符严格匹配**（无 trim 等宽松回退）；并通过 `mergeRanges` 区间合并后的「完整读取覆盖」检查：**读过 10–12 行，就不能引用 1–100 行**。该检查封堵了「模型只读

过几行却给出大段引文」的幻觉通道（R04 任务修复的核心漏洞，含专项回归测试）。

- **规范引用校验**——只能引用本轮 `retrieve_guidelines` 实际返回的 `chunkId`（`retrievedChunkIds` 白名单）；「可访问但本轮未返回」同样拒绝，防止模型背诵规范条目伪造检索依据。
- **无效处理**——无效的 `primary` / `related` / 规范引用记录原因，不静默认可；结构损坏最多 1 次修复轮后丢弃；无效计数以 `finding.invalid` 事件单列，**不静默丢弃、不参与美化指标**（评测口径中无效引文候选计入 FP）。

4.5 低置信度待核查与风险指数

模型自评 `confidence < 0.7` 或触发条件不完整的问题进入 `needs_review`（待核查）状态：不进入高可信风险汇总、**不计入风险指数**，但绝不静默丢弃。风险指数公式为 `min(100, 20×critical + 10×high + 4×medium + low)`；用户「确认问题 / 标记误报」的反馈会即时重算。来源标签 `static` / `ai` / `combined` 分列展示；`valid` 只代表引文有效，不代表漏洞已被证实——这层区分在界面与报告中始终保留。

4.6 工具执行轨迹

每次工具调用持久化到 `tool_calls`（sequence、工具名、输入摘要、**真实计时的** `elapsedMs`、状态），界面「工具轨迹」面板仅展示这四项与成功状态——**不展示、也不索要模型内部思维链**；`budget_exceeded` 等异常状态如实传递。用户因此可以审计「AI 到底读了什么、花了多久」。

4.7 预算与限流（全部服务端强制）

维度	扫描	追问	补丁提案
模型请求	≤12（含修复与重试）	≤4	≤2
工具调用	≤30	≤8	—
token	输入 ≤60,000 / 输出 ≤8,000	—	—
墙钟	180s	60s	60s
单请求超时	30s	30s	30s

日额度（默认 `AI_DAILY_TOKEN_LIMIT=300000`）在每次调用**前**经单条原子条件 UPDATE 预留（`used + reserved + n ≤ 上限`，并发安全，20 并发预留集成测试不超限）；调用失败回收预留，结算实测优先、缺失回退保守估算（中日韩约 1 token/字，其余 4 字符/token），实测与估算分列展示。达到任一上限保存已有有效结果并标 `partial`，不冒充 `completed`；Mock provider 不预留、不记账。另有访问码失败限流（每 IP 每 10 分钟 10 次）与每会话最多一个活动扫描。

4.8 受限追问与可审阅补丁

- **追问 (F07)** ——归属 guard 限制在当前会话当前问题（跨会话 / 预置项目一律 404）；用户消息先持久化再处理；回答经同一证据校验器；证据不足显式返回 `insufficient_evidence`；消息按 `(created_at, id)` 复合游标分页。
- **补丁 (F08)** ——模型经 `submit_patch` 工具提交单文件编辑；服务端校验链全部独立执行：`baseFileHash` 一致、`expectedOldText` 严格匹配、行范围合法且互不重叠、 ≤ 200 行、命中脱敏区间拒绝导出、禁改锁文件与多文件。通过后在**内存副本**上生成 unified diff；语法比较仅标注「是否引入新错误」（TS Compiler API / @vue/compiler-sfc），测试恒为 `not_run` 并提示用户自行测试；无效提案 409 不可下载；快照原件字节不变（集成测试断言）。

5. 评测与可复现实验

总口径声明（页面与 CLI 固定展示）。 24 / 16 / 8 / 12 等小样本结果只称为「本样例集结果」，不能推广为真实项目准确率。**真实模型评测已于 2026-09-19 执行**（DeepSeek `deepseek-chat`，holdout 每模式重复 3 次，数字见 5.5 节）；5.4 节 llm 两行数字为 2026-09-13 受控 Mock 管线验证（结果逐项标注 `provider=mock`），仅验证管线与消融开关的正确性，两套数字分栏并存、互不覆盖。

5.1 数据集构成 (fixtures, 版本 v1-cd06ca6f)

24 个小型 TS/JS fixture 项目 = 12 缺陷 + 12 相近对照，覆盖 6 类问题 × 每类 2 场景；每个缺陷项目 manifest 记录 ruleId / category / file / 锚点 / symbol / 触发条件。生成器幂等（两次运行字节一致）且带**强校验**：标注 ↔ 静态规则输出双向对应、对照项目零发现、划分必须 16/8——校验失败即中止生成。开发集 16（8 缺陷 + 8 对照）/ 保留集 8（4 + 4）**按项目整体拆分**，文件不跨集；datasetVersion 以全部项目规范 JSON 的 sha256 前 8 位生成，内容变化 revision 自增，旧结果仍可按版本追溯。

问题类别	两个场景
动态代码执行	动态表达式求值 / new Function + 字符串定时器
HTML 注入入口	innerHTML / dangerouslySetInnerHTML
跨窗口消息源	targetOrigin="*" / 监听无 origin 校验
JSX 列表缺失 key	map 无 key / Fragment 子元素无 key
Hook 条件 / 非法作用域调用	if 内 useState / 普通函数内 useEffect

问题类别	两个场景
检查禁用与调试残留	裸 @ts-ignore / console 残留 + 禁用安全 ESLint 规则

5.2 三模式配置与执行口径

- **三模式**—— `static_only`：关闭云端 AI，纯静态规则，零模型调用； `llm_no_rag`：开启 AI，经 provider 包装层**移除** `retrieve_guidelines` **工具**（消融对照组，同模型 / prompt / 预算 / 工具，仅切换规范检索）； `hybrid_rag`：完整双路检索管线。
- **与生产同管线**——评测逐项目执行与生产完全一致的核心路径：构造真实 ZIP → 生产导入（含脱敏）→ 持久化快照 → 扫描任务 → 评测执行器直接领取租约执行六阶段；项目落在隔离预置项目，不借用其他会话快照。
- **入口与可复现**——页面 `/evaluation`（仅 admin 可触发，demo 会话如实 403）或 CLI `pnpm eval -- --mode hybrid_rag --split dev`；完整配置（数据集版本、划分、规则版本 `static-rules-v1`、provider、projectLimit）写入 `evaluations.config.json` 与 `metrics_json.config`。

5.3 指标定义（不通过丢弃美化指标）

候选兼容 = 文件一致 + 标注行段与报警行段相交 + 类别一致；一对一**确定性贪心匹配**（边按「起点距离 → 候选键序 → 标注键序」稳定排序，同输入同配对）；重复报警（兼容但标注已被占用）额外计 FP；**无效引文候选不参与匹配、单列 invalidCitations 且计入 FP**；Precision / Recall / F1 为调和平均，分母为零返回 null（页面显示 N/A，不显示 0 或 100%）；另报证据有效率、延迟 p50 / p95（percentile 线性插值）与 token 用量（实测与估算分列）；跨项目先求和 TP/FP/FN 再算派生指标。指标计算由 18 项手算表单元测试锁定（全对 / 漏报 / 误报 / 类别不符 / 零分母 N/A / 重复报警 FP / 无效引文 FP / 匹配确定性 / 聚合口径）。

5.4 受控评测结果（2026-09-13：静态基线 + Mock 管线验证）

运行模式	项目	TP / FP / FN	P / R / F1	证据有效率	延迟 p50 / p95	token / 项目	结果类
static_only · dev	16	9 / 0 / 0	1.000 / 1.000 / 1.000	100%	33ms / 66ms	0	真实无模型
static_only · holdout	8	6 / 0 / 0	1.000 / 1.000 / 1.000	100%	40ms / 70ms	0	真实无模型
llm_no_rag · dev	16	9 / 0 / 0	1.000 / 1.000 / 1.000	100%	49ms / 75ms	968 (合计 15,492)	Mock provider
hybrid_rag · dev	16	9 / 0 / 0	1.000 / 1.000 / 1.000	100%	52ms / 75ms	1,848 (合计 29,560)	Mock provider

表：三模式受控评测结果（Mock 管线验证）。

来源 docs/evaluation.md R08 记录 (2026-09-13 时点, 数字保留不动) ; static_only 两行为确定性真实静态规则结果, llm 两行为受控 Mock 管线; 每个数字可经评测页逐项目「查看」跳转 /scans/:id 回溯原始扫描。真实模型实测数字见 5.5 节。

- **消融可回溯验证**——hybrid_rag 运行 (16 项目) 逐项目 scanId 共产生 24 条 scan_citations 规范引用快照; llm_no_rag 对应运行 0 条: RAG 开关严格生效, 而非纸面声明。
- **static_only 满分的含义**——样例集与现有 9 条静态规则对齐 (生成器强校验保证标注 ↔ 规则输出双向对应), 该数字只说明「管线 + 规则在**本样例集**上的行为符合设计预期」, 不代表真实项目准确率。
- **Mock 语义**——llm 模式的 token / 延迟为 Mock 执行的管线口径, 不反映真实模型成本与延迟; Mock 的满分 precision 反映受控管线行为, 不冒充真实模型指标。规格 13 要求的真实模型保留集重复 3 次 (均值与范围) 评测已于 2026-09-19 执行, 见 5.5 节。

5.5 真实模型评测结果 (2026-09-19, DeepSeek, CLI 真实运行)

规格 13 要求的保留集重复 3 次 (均值与范围) 真实模型评测: 6 次运行 (llm_no_rag ×3 + hybrid_rag ×3, holdout 8 项目 / 次) **全部 completed**, 零失败项目、零崩溃重跑; evaluation_projects 48 行 provider=openai、provider_is_mock=false、model_id=deepseek-chat ——确认真实调用, 非 Mock。逐次明细、6 条 evaluations 记录 ID 与逐项目 scanId 见仓库 deliverables/real-ai-holdout-2026-09-19.md 。

- **环境**——deepseek-chat (DeepSeek OpenAI 兼容模式, 实测映射 deepseek-flash) ; 数据集 v1-cd06ca6f , 划分 holdout 8 项目 (4 缺陷 + 4 对照) ; CLI 执行器, 本地 PGlite socket, 预算受 AI_DAILY_TOKEN_LIMIT 约束。
- **词法检索降级声明**——DeepSeek 无 embedding API, hybrid_rag 的向量检索路降级为纯词法检索: 本节数字口径为「词法检索 × 真实模型」, pgvector 混合检索的真实效果未验证。

指标 (holdout 8 项目 × 3 次均值 ± 范围 min-max)	llm_no_rag	hybrid_rag
Precision	0.396 (0.353–0.462)	0.421 (0.400–0.462)
Recall	1.000 (三次全满)	1.000 (三次全满)
F1	0.566 (0.522–0.632)	0.591 (0.571–0.632)
证据有效率	75.5% (69.2–82.4%)	79.7% (73.3–92.3%)
延迟 p50 / p95	8,025 / 10,113ms	7,389 / 10,074ms
token / 项目	10,137 (9,201–11,051)	13,025 (11,561–14,787)

表：真实模型两模式均值 ± 范围 (n=3) 。

来源 docs/evaluation.md §4.5 与 deliverables/real-ai-holdout-2026-09-19.md ; 聚合口径为跨项目先求和 TP/FP/FN 再派生。

- **RAG 消融可回溯验证**——llm_no_rag 三次 scan_citations 均为 0 条（RAG 隔离严格生效）；hybrid_rag 三次 5 / 9 / 9 条，命中的规范均为预置库真实标题（与 holdout 缺陷类别对应），并非纸面声明。

维度	Mock (5.4 节, dev 16 项目)	真实模型 (本节, holdout 8 项目 × 3 均值)
llm_no_rag token / 项目	968	10,137 (≈10.5×)
hybrid_rag token / 项目	1,848	13,025 (≈7.0×)
延迟	49–75ms	p50 7,389–8,025ms / p95 10,074–10,113ms
P / R / F1 (llm 两模式)	1.000 / 1.000 / 1.000	P 0.396–0.421 / R 1.000 / F1 0.566–0.591
scan_citations (hybrid)	24 条 / 16 项目	23 条 / 3×8 项目
provider	mock	openai (deepseek-chat→deepseek-flash)

表：Mock 与真实模型分栏对照。

Mock 的满分 precision 反映受控管线行为；真实模型在对照项目上的误报是两者差异的主因。

- **如实结论**——Recall 三次全满（6/6 标注全部命中）；Precision 为短板，FP 主因为 fx-ctl-* 对照项目误报（模型在「相近对照」上仍报出告警）。RAG 带来 P +2.5pp（0.396→0.421）、F1 +2.5pp、证据有效率 +4.2pp，方向一致但幅度在小样本方差内，不足以宣称显著。规格 13 目标（holdout hybrid_rag P ≥ 0.80、R ≥ 0.65）：**R 达标 (1.000)、P 未达标 (0.421)**——目标是目标而非承诺，如实记录，不做 extrapolation。
- **已知异常**——fx-ctl-jsx-02 六次扫描终态均为 partial（指标正常产出并计入结果，原因待排查）；无效引文每次 1–4 条，按设计计入 FP 不丢弃。
- **token 总消耗**——555,892（llm_no_rag 243,296 + hybrid_rag 312,596），当日额度上限 AI_DAILY_TOKEN_LIMIT=2,000,000，余量充足。

6. 前端工程质量

6.1 测试策略：三层全绿

层级	规模 (2026-09-15 实测)	覆盖重点 (示例)
单元 (Vitest)	14 文件 / 197 用例	证据校验回归 17 项（读 10–12 行不能引用 1–100、严格引文、区间合并）；预算估算与上限 11 项；评测指标手算表 18 项；fixtures 构成校验 14 项；9 条静态规则每条 ≥2 命中 + ≥2 不命中；导入路径 / 脱敏 / 归档防护；演示语料契约 6 项

层级	规模 (2026-09-15 实测)	覆盖重点 (示例)
集成 (Vitest)	14 文件 / 127 用例	真实数据库上的租约故障恢复 9 场景 (续租保持 / 旧 worker 接管后不能写 / 取消中断 / 重试上限 / 收割 / 阶段提交后崩溃恢复 / 活动删除 / TTL 并发 / 失租 abort) ; AI 审查完整链路与日额度 20 并发原子预留; 评测 worker 端到端与消融验证 (no_rag 引用=0 / hybrid 引用>0) ; 导入、报告、补丁、会话隔离与越权 404
E2E (Playwright)	8 用例 / 约 49 秒	完整用户闭环 (导入 → 扫描 → 证据 → 反馈 → 补丁 → 导出报告 → 二次导入 → 对比四类变化断言) ; SSE 断开降级轮询并到达终态; 跨会话访问一律 404; 评测中心权限与指标非伪造; 1440×900 与 390×844 无横向溢出

E2E 运行于隔离环境: 独立构建目录、临时 PGLite / 存储 / fixtures、生产构建 + worker、`AI_PROVIDER=mock`、与开发端口隔离, 不触碰开发数据。

全量检查命令的真实退出码记录于 `docs/release-checklist.md` (2026-09-15) : `typecheck` (`tsc --noEmit, strict`) = 0、`lint` (`max-warnings 0`) = 0、`test:unit` = 0、`test:integration` = 0、`test:e2e` = 0、`build` = 0、`fixtures:generate` = 0、`doctor` = 0 (自检不输出任何密钥) 。

6.2 类型与契约

TypeScript strict 全量检查 ; Zod 合同层 (`findings / patch / scan / conversation / evaluation / api`) 同时约束模型输出与 API 边界 ; 界面全部纯文本渲染 (无 `innerHTML / dangerouslySetInnerHTML`) ; 导出 HTML 动态内容白名单净化, Markdown 用户内容全转义防注入。

6.3 体验工程

- **首屏与恢复**——服务端组件预取直出扫描页 / 问题详情页首屏; 刷新从持久化结果恢复终态, 不依赖内存状态。
- **实时性**——SSE 断线按 Last-Event-ID 续传 + 15s 心跳; 连续失败自动降级轮询 (2s 起步退避至 10s 封顶) , 恢复后停止; 该降级路径有专门 E2E 用例。
- **响应式与可访问性**——优先保证 1440×900 / 1024×768 / 390×844 三视口 (E2E 断言无横向溢出) ; 1024px 以下问题详情右栏进入「问题 / 代码 / 解释」标签; 390px 折叠导航含 `aria-expanded`、Escape 收起、路由变化自动收起 (16 项交互探针全部通过) ; 截图脚本自动产出 7 页面 × 3 视口共 21 张验证图。
- **状态诚实**——排队 / 进行中 / 局部完成 (partial) / 取消 / 失败状态完整; 每条问题以 severity / category / 来源 / 证据状态 / 人工反馈多维呈现, 不只靠颜色区分; 模型按钮均体现额度耗尽、限流、网络失败状态。

7. 创新性与差异点

7.1 与既有方案的对比

方案	典型局限	CodeAtlas 的回应
整仓发送给模型	成本高，覆盖不透明，输出不稳定	风险优先抽样 ≤30 文件 + 预算前置 + 静态 / AI 覆盖单独计数
纯静态规则工具	解释不贴近上下文，难判优先级	规则打底 + AI 定向读取 + 风险指数排序 + needs_review 分层
通用聊天工具	手工整理代码，引用难核实，不可复现	每条结论路径 / 行号 / 引文可点击核验，服务端独立校验
多智能体自主修改并运行仓库	需执行沙箱、权限隔离与昂贵调度	首版不采用；四件只读工具合同保持简单可控，未来可接 MCP

7.2 核心创新点

- 证据定位 + 完整读取覆盖校验**——不止匹配引文，还要求引用行段被模型**真实读取覆盖**（mergeRanges 区间合并），堵住「读过几行却引用整段」的幻觉通道；无效证据单列计数，绝不静默丢弃。
- 检索来源可追溯**——版本化规范库（contentHash）+ pgvector/字法双路 RRF 检索 + 规范引用白名单 + `scan_citations` 引用快照：知识库变更后旧报告仍可解释当时依据。
- 可审阅补丁而非自动修复**——单文件 diff + 服务端四重校验链 + 测试恒 `not_run` + 不写入原件：把修复的最终决定权与验证责任明确留给用户。
- 诚实标签体系**——static / ai / combined 来源、valid / needs_review 证据状态、Mock / 真实模型、「未再检出 ≠ 已验证修复」：把「AI 说的」与「系统验证的」在界面上系统性分开。
- 不可信数据防护**——代码注释、README、规范文档、用户问题均按待审数据处理：系统提示禁止指令注入扩大权限，工具参数服务端校验，模型输出 Zod 校验，界面纯文本渲染。
- 四态快照对比**——匹配键不依赖行号（规则 / 类别 + 规范化路径 + 符号名 + 去空白局部代码哈希，双路 64 位确定哈希），未覆盖文件归入「不可比较」而非「未再检出」，避免虚假的修复结论。

7.3 社会价值与适用场景

最直接的受益者是课程设计、竞赛项目与首个 Web 产品交付前的学生自查：证据链不只是结论，也是教学材料——「为什么这里是问题」以可点击的代码行与规范出处呈现。对小团队而言，系统是无专职审查人员时的补充防线。诚实边界同时是信任基础：工具不夸大能力，才敢被用于教学与自查。潜在延展方向包括高校实验室 / 培训机构的内网部署（两级访问码、会话隔离与 TTL 清理已具

备基本形态)、按预算接入真实模型 (AI SDK 路径已实现并经 2026-09-19 DeepSeek 真实评测验证), 以及 P1 规划的只读 GitHub 仓库导入; 本项目不宣称已具备商业化验证。

8. 部署与使用

8.1 本地快速开始 (PGLite, 无需 Docker)

```
pnpm install
pnpm db:dev
pnpm db:migrate && pnpm seed && pnpm doctor
pnpm dev
pnpm worker:dev
pnpm fixtures:generate
```

终端 1: 本地数据库 (PGLite socket, 端口 5432)

终端 2: 迁移 + 预置 30 条规范 + 自检

终端 2: Web 应用 `http://localhost:3100`

终端 3: worker (扫描 / 索引 / 评测后台)

评测前置: 生成 24 样例数据集 (幂等)

打开 `http://localhost:3100` 输入演示访问码进入工作台。默认访问码与 `.env.example` 一致 (`codeatlas-demo` / `codeatlas-admin`, admin 才能触发评测), **仅限本地演示, 生产必须更换**。本地 AI 默认 `AI_PROVIDER=mock`: 界面所有 AI 输出 (审查结论 / 追问回答 / 补丁提案) 都带「Mock (非真实模型)」标注。

8.2 关键环境变量

变量	用途
<code>DATABASE_URL</code>	PostgreSQL 连接串: 本地指向 PGLite socket 服务, 生产指向 compose 的 db
<code>STORAGE_ROOT</code>	快照与上传文件的持久化根目录 (默认 <code>.data/storage</code>)
<code>SESSION_SECRET</code> / <code>DEMO_ACCESS_CODE</code> / <code>ADMIN_ACCESS_CODE</code>	会话签名密钥与两级访问码; 生产必须从环境变量注入, 禁止默认值
<code>AI_PROVIDER</code>	<code>mock</code> (本地演示) 或 <code>openai</code> (OpenAI 兼容接口)
<code>AI_BASE_URL</code> / <code>AI_API_KEY</code> / <code>AI_CHAT_MODEL</code>	真实模型接入 (密钥不写入日志与消息)
<code>AI_EMBEDDING_MODEL</code> / <code>EMBEDDING_DIM</code>	规范向量化模型与维度 (缺省时检索降级词法并标注 <code>lexical_only</code>)
<code>AI_DAILY_TOKEN_LIMIT</code>	全局日 token 预算 (默认 300,000; 调用前原子预留, 达上限降级静态扫描)
<code>DATA_TTL_HOURS</code>	会话与项目数据过期小时数 (默认 24, worker 每小时清理)
<code>INTEGRATION_DATABASE_URL</code>	集成测试直连真实 PostgreSQL (库名须含 <code>test</code> 、账号需 <code>CREATEDB</code> ; 逐测试文件建 / 删独立临时库)

完整清单 (18 项, 含示例值) 见仓库 README「环境变量」; 真实密钥只存在于服务端进程与本机 `.env` (不入库)。

8.3 生产路径 (PostgreSQL + Docker Compose)

`compose.yaml` 提供 db (`pgvector/pgvector:pg16` + 健康检查) → 一次性 migrate / seed 服务 → web / worker 三个服务, 共享 storage 持久卷; `Dockerfile` 多阶段构建、非 root 运行 (USER node)、不挂载 Docker socket; 评测数据集 fixtures 随镜像分发 (COPY 缺失会使构建直接失败而非产出「数据集未就绪」的镜像)。生产强制从环境变量注入 `SESSION_SECRET` 与访问码; 公网暴露前应放置 HTTPS 反向代理 (生产 cookie 开启 Secure)。首次部署验证项: 向测试专用 PostgreSQL 设置 `INTEGRATION_DATABASE_URL` 后运行 `pnpm test:integration`, 在生产 PostgreSQL 上重跑租约 / 额度原子性测试。

8.4 已知限制 (诚实清单)

- 真实模型评测已执行 (2026-09-19)**: DeepSeek `deepseek-chat` holdout 8 项目 × 3 次/模式共 6 次运行全部 completed (数字见 5.5 节), 真实 provider 代码路径 (AI SDK v5、Chat Completions、30s 超时 / 180s 墙钟 / 日额度原子预留 / 证据校验) 经受真实网络调用验证。遗留限制: DeepSeek 无 embedding API, `hybrid_rag` 向量检索路降级为纯词法 (「pgvector 混合检索 × 真实模型」未验证); 复杂追问下模型易用满 8 次工具调用导致 `budget_exhausted` 无回答 (冒烟实测, 规格值按受控行为设计)。
- Docker / Compose 本地未验证**: 开发机无 Docker; 文件按规格编写并通过人工审阅, 未执行过 `docker build` / `docker compose up`。
- 生产 PostgreSQL 直连测试机制就绪但未实测**: 本机无 PostgreSQL 16 实例; 本地验证基线为 PGlite socket。
- 评测为 24 样例小样本口径**: 指标方差大, 只可作管线回归与演示, 不可作为产品能力声明。
- 2026-09-13 llm 模式数字为 Mock 管线**: 数字逐项标注 `provider=mock`, 仅验证管线与消融开关; 与 2026-09-19 真实模型数字 (5.5 节) 分栏并存、互不覆盖。
- 演示脚本中 AI 环节均为 Mock 演示**: 数字类演示来自真实静态评测记录。

参考与来源

本文档全部事实来自以下可回溯来源; 评测数字与测试统计为其中记录的真实命令运行结果。

- 公开仓库: github.com/aabbcc070403/codeatlas
- `README.md` —— 功能总览 F01-F12、快速开始、环境变量、安全边界、已知限制
- `docs/architecture.md` —— 模块图、数据流、安全边界、数据库合同、构建实测细节

- docs/evaluation.md ——数据集构成、三模式配置、指标定义、R08 受控数字（2026-09-13）与真实模型实测数字（\$4.5, 2026-09-19）
- deliverables/real-ai-holdout-2026-09-19.md ——真实模型（DeepSeek）holdout 3+3 逐次明细、汇总与 token 消耗记录
- docs/demo-script.md ——5-8 分钟演示脚本，逐段标注 Mock 环节与真实数字来源
- docs/progress.md ——T01-T08 与 R01-R09 全部执行记录（命令、退出码、实测结果）
- docs/release-checklist.md ——2026-09-15 全量检查命令真实退出码与阶段状态
- 《01-项目规格》——产品定位、技术路线选择与差异点背景（仅作参考）